Déployez des logiciels

BeeGFS on NetApp with E-Series Storage

NetApp
December 15, 2022

This PDF was generated from https://docs.netapp.com/fr-fr/beegfs/beegfs-deploy-setup-nodes.html on December 15, 2022. Always check docs.netapp.com for the latest.

Table des matières

Déployez des logiciels . . . 1
Configurez les nœuds de fichiers et les nœuds en mode bloc . . . 1
Configurez un nœud de contrôle Ansible . . . 2
Créez l’inventaire Ansible . . . 4
Définissez l’inventaire Ansible pour les éléments de base BeeGFS . . . 17
Déployez BeeGFS . . . 31
Configurer les clients BeeGFS . . . 34

Déployez des logiciels

Configurez les nœuds de fichiers et les nœuds en mode bloc

Si la plupart des tâches de configuration logicielle sont automatisées au moyen des collections Ansible fournies par NetApp, vous devez configurer la mise en réseau sur le contrôleur de gestion de la carte de base (BMC) de chaque serveur et configurer le port de gestion sur chaque contrôleur.

Configurez les nœuds de fichiers

1. Configurez la mise en réseau sur le contrôleur de gestion de la carte mère (BMC) de chaque serveur.

   Pour savoir comment configurer le réseau pour les nœuds de fichiers validés du serveur Lenovo SR665, consultez le "Documentation Lenovo ThinkSystem".

   > Un contrôleur de gestion de la carte mère (BMC), parfois appelé processeur de service, est le nom générique de la fonctionnalité de gestion hors bande intégrée dans diverses plates-formes de serveurs qui fournissent un accès à distance même si le système d'exploitation n'est pas installé ou accessible. Les fournisseurs vendent généralement cette fonctionnalité avec leur propre marque. Par exemple, sur le Lenovo SR665, le contrôleur BMC est appelé le contrôleur XClarity (XCC)_ de _Lenovo.

2. Configurez les paramètres du système pour des performances maximales.

   Vous configurez les paramètres système à l'aide de la configuration UEFI (anciennement appelée BIOS) ou en utilisant les API Redfish fournies par de nombreux BMCs. Les paramètres système varient en fonction du modèle de serveur utilisé comme nœud de fichier.

   Pour savoir comment configurer les paramètres système des nœuds de fichiers Lenovo SR665 validés, reportez-vous à la section "Réglez les paramètres du système en fonction des performances".

3. Installez Red Hat 8.4 et configurez le nom d'hôte et le port réseau utilisés pour gérer le système d'exploitation, y compris la connectivité SSH à partir du nœud de contrôle Ansible.

   Ne configurez pas d'adresses IP sur l'un des ports InfiniBand pour le moment.

   > Bien qu'il ne soit pas strictement nécessaire, les sections suivantes présument que les noms d'hôte sont numérotés séquentiellement (comme h1-HN) et font référence aux tâches qui doivent être effectuées sur les hôtes impairs et pairs.

4. Utilisez RedHat Subscription Manager pour enregistrer et abonner le système afin de permettre l'installation des packages requis dans les référentiels Red Hat officiels et de limiter les mises à jour vers la version prise en charge de Red Hat : `subscription-manager release --set=8.4`. Pour obtenir des instructions, reportez-vous à la section "Comment enregistrer et souscrire un système RHEL" et "Comment limiter les mises à jour".
5. Activez le référentiel Red Hat contenant les packages requis pour la haute disponibilité.

```
subscription-manager repo-override --repo=rhel-8-for-x86_64
-highavailability-rpms --add=enabled:1
```

6. Mettez à jour tous les micrologiciels de l'HCA ConnectX-6 avec la version recommandée dans "Exigences technologiques".

   Cette mise à jour peut être effectuée en téléchargeant et en exécutant une version de l'outil mlxup qui regroupe le micrologiciel recommandé. Vous pouvez télécharger cet outil à partir de "Mlxup - Utilitaire de mise à jour et de requête".

Configurez les nœuds en mode bloc

Configurez les nœuds en mode bloc EF600 en configurant le port de gestion sur chaque contrôleur.

1. Configurez le port de gestion sur chaque contrôleur EF600.

   Pour obtenir des instructions sur la configuration des ports, reportez-vous à la section "Centre de documentation E-Series".

2. Vous pouvez également définir le nom de la matrice de stockage pour chaque système.

   La définition d'un nom peut faciliter la référence à chaque système dans les sections suivantes. Pour obtenir des instructions sur la définition du nom de la matrice, accédez à "Centre de documentation E-Series".

> Bien qu'il ne soit pas strictement nécessaire, les rubriques suivantes présument que les noms des matrices de stockage sont numérotés de façon séquentielle (comme c1 - CN) et font référence aux étapes à suivre sur les systèmes pairs ou impairs.

Configurez un nœud de contrôle Ansible

Pour configurer un nœud de contrôle Ansible, vous devez identifier une machine virtuelle ou physique avec un accès réseau aux ports de gestion de tous les nœuds de fichiers et de blocs qui peuvent être utilisés pour configurer la solution.

Les étapes suivantes ont été testées sur CentOS 8.4. Pour connaître les étapes spécifiques à votre distribution Linux préférée, reportez-vous au "Documentation Ansible".

1. Installer Python 3.9 et s'assurer que la version correcte de `pip` est installé.

```
sudo dnf install python3.9 -y
sudo dnf install python39-pip
sudo dnf install sshpass
```

2. Créez des liens symboliques, en vous assurant que le binaire Python 3.9 est utilisé chaque fois `python3` ou `python` est appelé.

```
sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

3. Installez les packages Python requis par les collections NetApp BeeGFS.

```
python3 -m pip install ansible cryptography netaddr
```

> Pour vous assurer que vous installez une version prise en charge d'Ansible et de tous les packages Python requis, consultez le fichier Lisezmoi de la collection BeeGFS. Les versions prises en charge sont également présentées dans le "Exigences techniques".

4. Vérifiez que les versions correctes d'Ansible et de Python sont installées.

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. Stockez les inventaires Ansible utilisés pour décrire le déploiement BeeGFS dans des systèmes de contrôle source tels que Git ou BitBucket, puis installez Git pour interagir avec ces systèmes.

```
sudo dnf install git -y
```

6. Configurez SSH sans mot de passe. C'est le moyen le plus simple de permettre à Ansible d'accéder aux nœuds de fichiers BeeGFS distants à partir du nœud de contrôle Ansible.
   a. Sur le nœud de contrôle Ansible, si nécessaire, générez une paire de clés publiques à l'aide de `ssh-keygen`
   b. Configurez SSH sans mot de passe sur chacun des nœuds de fichiers à l'aide de `ssh-copy-id <ip_or_hostname>`

      Do **NOT** configurez SSH sans mot de passe sur les nœuds de bloc. Cela n'est ni pris en charge ni obligatoire.

7. Utilisez Ansible Galaxy pour installer la version de la collection BeeGFS répertoriée dans "Exigences techniques".

   Cette installation inclut des dépendances Ansible supplémentaires, telles que le logiciel NetApp SANtricity et les collections d'hôtes.

   ```
   ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
   ```

Créez l'inventaire Ansible

Pour définir la configuration des nœuds de fichiers et de blocs, vous créez un inventaire Ansible qui représente le système de fichiers BeeGFS que vous souhaitez déployer. L'inventaire inclut les hôtes, les groupes et les variables décrivant le système de fichiers BeeGFS souhaité.

Étape 1 : définir la configuration de tous les éléments de base

Définissez la configuration qui s'applique à tous les blocs de construction, quel que soit le profil de configuration que vous pouvez appliquer individuellement.

Avant de commencer

- Utilisez un système de contrôle source tel que BitBucket ou Git pour stocker le contenu du répertoire contenant les fichiers d'inventaire et de PlayBook Ansible.
- Créer un `.gitignore` Fichier qui spécifie les fichiers que Git doit ignorer. Cela évite de stocker des fichiers volumineux dans Git.

Étapes

1. Sur votre nœud de contrôle Ansible, identifiez un répertoire à utiliser pour stocker les fichiers d'inventaire et de PlayBook Ansible.

   Sauf indication contraire, tous les fichiers et répertoires créés dans cette étape et les étapes suivantes sont créés par rapport à ce répertoire.

2. Créez les sous-répertoires suivants :

   `host_vars`

   `group_vars`

   `packages`

Étape 2 : définir la configuration des nœuds de fichiers et de blocs individuels

Définissez la configuration qui s'applique aux nœuds de fichiers individuels et aux nœuds d'élément de base individuels.

1. Sous `host_vars/`, Créez un fichier pour chaque noeud de fichier BeeGFS nommé `<HOSTNAME>.yml` Avec le contenu suivant, en portant une attention particulière aux notes concernant le contenu à remplir pour les adresses IP de cluster BeeGFS et les noms d'hôte se terminant par des nombres impairs et

impairs.

Initialement, les noms d’interface de nœud de fichier correspondent à ce qui est répertorié ici (comme ib0 ou ibs1f0). Ces noms personnalisés sont configurés dans <a href="https://docs.netapp.com/fr-fr/beegfs/Étape 4 : définissez la configuration qui doit s&.html#8217;appliquer à tous les nœuds de fichiers">https://docs.netapp.com/fr-fr/beegfs/Étape 4 : définissez la configuration qui doit s&.html#8217;appliquer à tous les nœuds de fichiers</a>.

```
ansible_host: “<MANAGEMENT_IP>”
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

> Si vous avez déjà déployé le cluster BeeGFS, vous devez arrêter le cluster avant d'ajouter ou de modifier des adresses IP configurées de manière statique, y compris les adresses IP et IP du cluster utilisées pour NVMe/IB. Cette modification est nécessaire afin que ces modifications prennent effet correctement et ne perturbent pas les opérations du cluster.

2. Sous `host_vars/`, Créez un fichier pour chaque noeud de bloc BeeGFS nommé `<HOSTNAME>.yml` et remplissez-le avec le contenu suivant.

   Faites particulièrement attention aux remarques concernant le contenu à remplir pour les noms de matrices de stockage se terminant par des nombres pairs ou impairs.

   Pour chaque noeud de bloc, créez un fichier et spécifiez `<MANAGEMENT_IP>` Pour un des deux contrôleurs (généralement Un).

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

Étape 3 : définissez une configuration à appliquer à tous les nœuds de fichiers et de blocs

Vous pouvez définir une configuration commune à un groupe d'hôtes sous `group_vars` dans un nom de

fichier correspondant au groupe. Cela empêche de répéter une configuration partagée à plusieurs endroits.

Description de la tâche

Les hôtes peuvent se trouver dans plusieurs groupes et au moment de l'exécution, Ansible choisit les variables qui s'appliquent à un hôte donné en fonction de ses règles de priorité de variable. (Pour plus d'informations sur ces règles, consultez la documentation Ansible pour "Utilisation de variables".)

Les affectations hôte-groupe sont définies dans le fichier d'inventaire Ansible réel, créé à la fin de cette procédure.

Étape

Dans Ansible, vous pouvez définir n'importe quelle configuration que vous souhaitez appliquer à tous les hôtes dans un groupe appelé `All`. Créez le fichier `group_vars/all.yml` avec le contenu suivant :

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

Étape 4 : définissez la configuration qui doit s'appliquer à tous les nœuds de fichiers

La configuration partagée pour les nœuds de fichiers est définie dans un groupe appelé `ha_cluster`. Les étapes de cette section créent la configuration qui doit être incluse dans le `group_vars/ha_cluster.yml` fichier.

Étapes

1. En haut du fichier, définissez les valeurs par défaut, y compris le mot de passe à utiliser comme `sudo` utilisateur sur les nœuds de fichiers.

   ```
   ### ha_cluster Ansible group inventory file.
   # Place all default/common variables for BeeGFS HA cluster resources
   below.
   ### Cluster node defaults
   ansible_ssh_user: root
   ansible_become_password: <PASSWORD>
   eseries_ipoib_default_hook_templates:
     - 99-multihoming.j2 # This is required when configuring additional
   static IPs (for example cluster IPs) when multiple IB ports are in the
   same IPoIB subnet.
   # If the following options are specified, then Ansible will
   automatically reboot nodes when necessary for changes to take effect:
   eseries_common_allow_host_reboot: true
   eseries_common_reboot_test_command: "systemctl --state=active,exited |
   grep eseries_nvme_ib.service"
   ```

> En particulier pour les environnements de production, ne stockez pas de mots de passe en texte brut. Utilisez plutôt Ansible Vault (voir "Cryptage de contenu avec Ansible Vault") ou le `--ask-become-pass` option lors de l'exécution du manuel de vente. Si le `ansible_ssh_user` est déjà `root`, ensuite, vous pouvez omettre le `ansible_become_password`.

2. Vous pouvez également configurer un nom pour le cluster haute disponibilité (HA) et spécifier un utilisateur pour les communications intra-cluster.

   Si vous modifiez le schéma d'adressage IP privé, vous devez également mettre à jour le schéma par défaut `beegfs_ha_mgmtd_floating_ip`. Ceci doit correspondre à ce que vous configurez plus tard pour le groupe de ressources BeeGFS Management.

   Spécifiez un ou plusieurs e-mails qui doivent recevoir des alertes pour les événements du cluster à l'aide de `beegfs_ha_alert_email_list`.

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0         # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

> Tout en apparence redondant, `beegfs_ha_mgmtd_floating_ip` Est important lorsque vous faites évoluer le système de fichiers BeeGFS au-delà d’un seul cluster HA. Les clusters HA suivants sont déployés sans service de gestion BeeGFS et point supplémentaires sur le service de gestion fourni par le premier cluster.

3. Configurer un agent d’escrime. (Pour plus de détails, voir "Configurer l’escrime dans un cluster Red Hat haute disponibilité".) Le résultat suivant montre des exemples de configuration d’agents d’escrime courants. Choisissez l’une de ces options.

   Pour cette étape, gardez à l’esprit que :

   - Par défaut, l’escrime est activé, mais vous devez configurer un *agent* d’escrime.

- Le `<HOSTNAME>` spécifié dans le `pcmk_host_map` ou `pcmk_host_list` Doit correspondre au nom d'hôte dans l'inventaire Ansible.
- L'utilisation du cluster BeeGFS sans escrime n'est pas prise en charge, particulièrement en production. Cela permet de s'assurer que les services BeeGFS, y compris les dépendances de ressources comme les périphériques de bloc, basculent en raison d'un problème, il n'y a aucun risque d'accès simultané par plusieurs nœuds qui entraînent une corruption du système de fichiers ou tout autre comportement indésirable ou inattendu. Si l'escrime doit être désactivé, reportez-vous aux notes générales du guide de démarrage et de mise en place du rôle BeeGFS HA `beegfs_ha_cluster_crm_config_options["stonith-enabled"]` à faux dans `ha_cluster.yml`.
- Plusieurs dispositifs d'escrime au niveau des nœuds sont disponibles, et le rôle BeeGFS HA peut configurer n'importe quel agent d'escrime disponible dans le référentiel de package Red Hat HA. Si possible, utilisez un agent d'escrime qui fonctionne via l'alimentation sans coupure (UPS) ou l'unité de distribution de l'alimentation en rack (RPDU), Parce que certains agents d'escrime, tels que le contrôleur de gestion de la carte mère (BMC) ou d'autres dispositifs d'éclairage intégrés au serveur, peuvent ne pas répondre à la demande de clôture dans certains scénarios de panne.

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify "1".
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. Activez le réglage des performances recommandé dans le système d'exploitation Linux.

   Si de nombreux utilisateurs trouvent les paramètres par défaut des paramètres de performance qui fonctionnent généralement bien, vous pouvez également modifier les paramètres par défaut d'une charge de travail donnée. Ainsi, ces recommandations sont incluses dans le rôle BeeGFS, mais ne sont pas activées par défaut pour s'assurer que les utilisateurs connaissent le réglage appliqué à leur système de fichiers.

   Pour activer le réglage des performances, spécifiez :

```
### Performance Configuration:
beegfs_ha_enable_performance_tuning: True
```

5. (Facultatif) vous pouvez régler les paramètres d'ajustement des performances dans le système d'exploitation Linux selon vos besoins.

Pour obtenir une liste complète des paramètres de réglage disponibles que vous pouvez régler, consultez la section Réglages de la performance par défaut du rôle BeeGFS HA dans "E-Series site GitHub BeeGFS". Les valeurs par défaut peuvent être remplacées pour tous les nœuds du cluster dans ce fichier ou dans le `host_vars` fichier pour un nœud individuel.

6. Pour permettre une connectivité complète de 200 Go/HDR entre des nœuds de bloc et de fichier, utilisez le package Open Subnet Manager (OpenSM) de Mellanox Open Fabrics Enterprise distribution (MLNX_OFED). (La boîte de réception `opensm` package ne prend pas en charge la fonctionnalité de virtualisation nécessaire.) Bien que le déploiement à l'aide d'Ansible soit pris en charge, vous devez d'abord télécharger les packages souhaités vers le nœud de contrôle Ansible utilisé pour exécuter le rôle BeeGFS.

   a. À l'aide de `curl` Ou téléchargez les packages pour la version d'OpenSM répertoriée dans la section exigences technologiques du site web de Mellanox vers le `packages/` répertoire. Par exemple :

```
curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm

curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
```

   b. Remplissez les paramètres suivants dans `group_vars/ha_cluster.yml` (réglez les colis si nécessaire) :

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. Configurer le `udev` Règle pour assurer un mappage cohérent des identificateurs de port InfiniBand logiques aux périphériques PCIe sous-jacents.

   Le `udev` La règle doit être unique à la topologie PCIe de chaque plate-forme de serveur utilisée comme nœud de fichier BeeGFS.

   Utilisez les valeurs suivantes pour les nœuds de fichiers vérifiés :

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. (Facultatif) mettre à jour l'algorithme de sélection de cible de métadonnées.

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> Lors des tests de vérification, `randomrobin` Est généralement utilisé pour s'assurer que les fichiers de test étaient répartis de façon égale sur toutes les cibles de stockage BeeGFS pendant l'évaluation des performances (pour plus d'informations sur l'analyse comparative, consultez le site BeeGFS pour "Analyse comparative d'un système BeeGFS"). Avec une utilisation réelle, il est possible que les cibles numérotées soient plus rapidement que les cibles numérotées plus élevées. Omission `randomrobin` et il suffit d'utiliser la valeur par défaut `randomized` la valeur a été indiquée pour fournir de bonnes performances tout en utilisant toujours toutes les cibles disponibles.

Étape 5 : définir la configuration pour le nœud de bloc commun

La configuration partagée pour les nœuds de bloc est définie dans un groupe appelé `eseries_storage_systems`. Les étapes de cette section créent la configuration qui doit être incluse dans le `group_vars/ eseries_storage_systems.yml` fichier.

Étapes

1. Définissez la connexion Ansible sur local, indiquez le mot de passe système et spécifiez si les certificats SSL doivent être vérifiés. (Normalement, Ansible utilise SSH pour la connexion aux hôtes gérés, mais dans le cas des systèmes de stockage NetApp E-Series utilisés comme nœuds de bloc, les modules utilisent l'API REST pour la communication.) En haut du fichier, ajoutez ce qui suit :

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> La liste des mots de passe en texte clair n'est pas recommandée. Utilisez un coffre-fort Ansible ou fournissez le `eseries_system_password` Lors de l'exécution d'Ansible avec `--extra-vars`.

2. Pour assurer des performances optimales, installez les versions répertoriées pour les nœuds de bloc dans "Exigences techniques".

   Téléchargez les fichiers correspondants à partir du "Site de support NetApp". Vous pouvez les mettre à niveau manuellement ou les inclure dans le `packages/` Répertoire du nœud de contrôle Ansible, puis remplissez les paramètres suivants dans `eseries_storage_systems.yml` Pour la mise à niveau avec Ansible :

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. Téléchargez et installez la dernière version du micrologiciel de lecteur disponible pour les lecteurs installés dans vos nœuds de bloc à partir du "Site de support NetApp". Vous pouvez les mettre à niveau manuellement ou les inclure dans le `packages/` Répertoire du nœud de contrôle Ansible, puis remplissez les paramètres suivants dans `eseries_storage_systems.yml` Pour la mise à niveau avec Ansible :

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> Réglage `eseries_drive_firmware_upgrade_drives_online` à `false` Accélère la mise à niveau, mais ne doit pas être effectuée avant le déploiement de BeeGFS. En effet, ce paramètre nécessite l'arrêt de toutes les E/S des disques avant la mise à niveau afin d'éviter les erreurs d'application. Bien que la mise à niveau en ligne du micrologiciel des lecteurs avant la configuration des volumes soit toujours rapide, nous vous recommandons de toujours définir cette valeur sur `true` pour éviter tout problème par la suite.

4. Pour optimiser les performances, effectuez les modifications suivantes de la configuration globale :

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. Pour optimiser le provisionnement et le comportement des volumes, spécifiez les paramètres suivants :

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> La valeur spécifiée pour `eseries_storage_pool_usable_drives` Est spécifique aux nœuds de bloc NetApp EF600 et contrôle l'ordre dans lequel les disques sont affectés aux nouveaux groupes de volumes. Cette commande permet de s'assurer que les E/S de chaque groupe sont réparties de manière homogène entre les canaux des disques back-end.

Définissez l'inventaire Ansible pour les éléments de base BeeGFS

Après avoir défini la structure d'inventaire générale Ansible, définissez la configuration de chaque élément de base dans le système de fichiers BeeGFS.

Ces instructions de déploiement montrent comment déployer un système de fichiers composé d'un élément de base, incluant la gestion, les métadonnées et les services de stockage, un deuxième élément de base avec des métadonnées et des services de stockage, et un troisième élément de base uniquement dédié au stockage.

Ces étapes sont destinées à afficher la gamme complète des profils de configuration standard que vous pouvez utiliser pour configurer les éléments de base NetApp BeeGFS de façon à répondre aux exigences du système de fichiers global BeeGFS.

Dans les sections suivantes et ceci, ajustez selon les besoins pour générer l'inventaire représentant le système de fichiers BeeGFS que vous voulez déployer. Utilisez notamment des noms d'hôte Ansible qui représentent chaque nœud de bloc ou de fichier et le schéma d'adressage IP souhaité pour le réseau de stockage, afin de vous assurer qu'il peut évoluer jusqu'au nombre de nœuds de fichiers et de clients BeeGFS.

Étape 1 : créez le fichier d'inventaire Ansible

Étapes

1. Créer un nouveau `inventory.yml` file, puis insérez les paramètres suivants en remplaçant les hôtes sous `eseries_storage_systems` si nécessaire pour représenter les nœuds en mode bloc dans votre déploiement. Les noms doivent correspondre au nom utilisé pour `host_vars/<FILENAME>.yml`.

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

Dans les sections suivantes, vous allez créer des groupes Ansible supplémentaires sous `ha_cluster` Qui représentent les services BeeGFS que vous voulez exécuter dans le cluster.

Étape 2 : configurer l'inventaire d'un élément de base de gestion, de métadonnées et de stockage

Le premier élément de base ou du cluster doit inclure le service de gestion BeeGFS ainsi que les services de métadonnées et de stockage :

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous `ha_cluster: children`:

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
```

```
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
```

```
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. Créez le fichier `group_vars/mgmt.yml` et inclure les éléments suivants :

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `meta_01` à `meta_08` à l'aide du modèle suivant, puis remplissez les valeurs des espaces réservés pour chaque service faisant référence au tableau ci-dessous :

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

La taille du volume est indiquée sous forme de pourcentage du pool de stockage global (également appelé groupe de volumes). NetApp recommande fortement de laisser une certaine capacité libre dans chaque pool afin d'autoriser le sur-provisionnement SSD (pour plus d'informations, voir "Présentation de la baie NetApp EF600"). Le pool de stockage, `beegfs_m1_m2_m5_m6`, alloue également 1% de la capacité du pool pour le service de gestion. Ainsi, pour les volumes de métadonnées dans le pool de stockage, `beegfs_m1_m2_m5_m6`, Si vous utilisez des disques de 1,92 To ou 3,84 To, définissez cette valeur sur `21.25`; Pour les lecteurs 7,65 To, définissez cette valeur sur `22.25`; Et pour les disques de 15,3 To, définissez cette valeur sur `23.75`.

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_01.yml	8015	i1b:100.127.101.1/16 i2b:100.128.102.1/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_02.yml	8025	i2b:100.128.102.2/16 i1b:100.127.101.2/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B
meta_03.yml	8035	i3b:100.127.101.3/16 i4b:100.128.102.3/16	1	ictad22a02	beegfs_m3_m4_m7_m8	A

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_04.yml	8045	i4b:100.128.102.4/16 i3b:100.127.101.4/16	1	ictad22a02	beegfs_m3_m4_m7_m8	B
meta_05.yml	8055	i1b:100.127.101.5/16 i2b:100.128.102.5/16	0	ictad22a01	beegfs_m1_m2_m5_m6	A
meta_06.yml	8065	i2b:100.128.102.6/16 i1b:100.127.101.6/16	0	ictad22a01	beegfs_m1_m2_m5_m6	B
meta_07.yml	8075	i3b:100.127.101.7/16 i4b:100.128.102.7/16	1	ictad22a02	beegfs_m3_m4_m7_m8	A
meta_08.yml	8085	i4b:100.128.102.8/16 i3b:100.127.101.8/16	1	ictad22a02	beegfs_m3_m4_m7_m8	B

4. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_01` à `stor_08` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_01.yml	8013	i1b:100.127.103.1/16 i2b:100.128.104.1/16	0	ictad22a01	beegfs_s1_s2	A
stor_02.yml	8023	i2b:100.128.104.2/16 i1b:100.127.103.2/16	0	ictad22a01	beegfs_s1_s2	B
stor_03.yml	8033	i3b:100.127.103.3/16 i4b:100.128.104.3/16	1	ictad22a02	beegfs_s3_s4	A
stor_04.yml	8043	i4b:100.128.104.4/16 i3b:100.127.103.4/16	1	ictad22a02	beegfs_s3_s4	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_05.yml	8053	i1b:100.127.103.5/16 i2b:100.128.104.5/16	0	ictad22a01	beegfs_s5_s6	A
stor_06.yml	8063	i2b:100.128.104.6/16 i1b:100.127.103.6/16	0	ictad22a01	beegfs_s5_s6	B
stor_07.yml	8073	i3b:100.127.103.7/16 i4b:100.128.104.7/16	1	ictad22a02	beegfs_s7_s8	A
stor_08.yml	8083	i4b:100.128.104.8/16 i3b:100.127.103.8/16	1	ictad22a02	beegfs_s7_s8	B

Étape 3 : configurer l'inventaire d'un élément de base métadonnées + stockage

Elles expliquent comment configurer un inventaire Ansible pour un élément de base de stockage + de métadonnées BeeGFS.

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous la configuration existante :

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
          hosts:
            ictad22h03:
```

```
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
            ictad22h03:
```

2. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `meta_09` à `meta_16` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_09.yml	8015	i1b:100.127.101.9/16 i2b:100.128.102.9/16	0	ictad22a03	beegfs_m9_m10_m13_m14	A
meta_10.yml	8025	i2b:100.128.102.10/16 i1b:100.127.101.10/16	0	ictad22a03	beegfs_m9_m10_m13_m14	B
meta_11.yml	8035	i3b:100.127.101.11/16 i4b:100.128.102.11/16	1	ictad22a04	beegfs_m11_m12_m15_m16	A
meta_12.yml	8045	i4b:100.128.102.12/16 i3b:100.127.101.12/16	1	ictad22a04	beegfs_m11_m12_m15_m16	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
meta_13.yml	8055	i1b:100.127.101.13/16 i2b:100.128.102.13/16	0	ictad22a03	beegfs_m9_m10_m13_m14	A
meta_14.yml	8065	i2b:100.128.102.14/16 i1b:100.127.101.14/16	0	ictad22a03	beegfs_m9_m10_m13_m14	B
meta_15.yml	8075	i3b:100.127.101.15/16 i4b:100.128.102.15/16	1	ictad22a04	beegfs_m11_m12_m15_m16	A
meta_16.yml	8085	i4b:100.128.102.16/16 i3b:100.127.101.16/16	1	ictad22a04	beegfs_m11_m12_m15_m16	B

3. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_09` à `stor_16` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage"..

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_09.yml	8013	i1b:100.127.103.9/16 i2b:100.128.104.9/16	0	ictad22a03	beegfs_s9_s10	A
stor_10.yml	8023	i2b:100.128.104.10/16 i1b:100.127.103.10/16	0	ictad22a03	beegfs_s9_s10	B
stor_11.yml	8033	i3b:100.127.103.11/16 i4b:100.128.104.11/16	1	ictad22a04	beegfs_s11_s12	A
stor_12.yml	8043	i4b:100.128.104.12/16 i3b:100.127.103.12/16	1	ictad22a04	beegfs_s11_s12	B
stor_13.yml	8053	i1b:100.127.103.13/16 i2b:100.128.104.13/16	0	ictad22a03	beegfs_s13_s14	A
stor_14.yml	8063	i2b:100.128.104.14/16 i1b:100.127.103.14/16	0	ictad22a03	beegfs_s13_s14	B
stor_15.yml	8073	i3b:100.127.103.15/16 i4b:100.128.104.15/16	1	ictad22a04	beegfs_s15_s16	A
stor_16.yml	8083	i4b:100.128.104.16/16 i3b:100.127.103.16/16	1	ictad22a04	beegfs_s15_s16	B

Étape 4 : configurer l'inventaire pour un élément de base stockage uniquement

Procédure de configuration d'un inventaire Ansible pour un élément de base BeeGFS Storage uniquement. La différence majeure entre l'installation de la configuration pour un bloc de métadonnées + stockage et un bloc modulaire uniquement destiné au stockage, c'est l'omission de tous les groupes de ressources de métadonnées et la modification `criteria_drive_count` de 10 à 12 pour chaque pool de stockage.

Étapes

1. Dans `inventory.yml`, remplissez les paramètres suivants sous la configuration existante :

```
    # ictad22h05/ictad22h06 HA Pair (storage only building block):
      stor_17:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_18:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_19:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_20:
        hosts:
          ictad22h05:
          ictad22h06:
      stor_21:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_22:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_23:
        hosts:
          ictad22h06:
          ictad22h05:
      stor_24:
        hosts:
          ictad22h06:
          ictad22h05:
```

2. Sous `group_vars/`, créez des fichiers pour les groupes de ressources `stor_17` à `stor_24` à l'aide du modèle suivant, puis remplissez les valeurs de paramètre fictif pour chaque service référençant l'exemple :

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

Pour connaître la taille correcte à utiliser, reportez-vous à la section "Pourcentages de surprovisionnement recommandés pour le pool de stockage".

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_17.yml	8013	i1b:100.127.103.17/16 i2b:100.128.104.17/16	0	ictad22a05	beegfs_s17_s18	A
stor_18.yml	8023	i2b:100.128.104.18/16 i1b:100.127.103.18/16	0	ictad22a05	beegfs_s17_s18	B
stor_19.yml	8033	i3b:100.127.103.19/16 i4b:100.128.104.19/16	1	ictad22a06	beegfs_s19_s20	A
stor_20.yml	8043	i4b:100.128.104.20/16 i3b:100.127.103.20/16	1	ictad22a06	beegfs_s19_s20	B

Nom du fichier	Port	Adresses IP flottantes	Zone NUMA	Nœud de bloc	Pool de stockage	Contrôleur propriétaire
stor_21.yml	8053	i1b:100.127.103.21/16 i2b:100.128.104.21/16	0	ictad22a05	beegfs_s21_s22	A
stor_22.yml	8063	i2b:100.128.104.22/16 i1b:100.127.103.22/16	0	ictad22a05	beegfs_s21_s22	B
stor_23.yml	8073	i3b:100.127.103.23/16 i4b:100.128.104.23/16	1	ictad22a06	beegfs_s23_s24	A
stor_24.yml	8083	i4b:100.128.104.24/16 i3b:100.127.103.24/16	1	ictad22a06	beegfs_s23_s24	B

Déployez BeeGFS

Un déploiement et une gestion de la configuration impliquent d'exécuter un ou plusieurs playbooks contenant les tâches Ansible requises pour exécuter et placer le système global dans l'état souhaité.

Même si toutes les tâches peuvent être incluses dans un seul manuel de vente, il est difficile pour les systèmes complexes de gérer cette tâche très rapidement. Ansible vous permet de créer et de distribuer des rôles comme un moyen de packaging des playbooks réutilisables et du contenu associé (par exemple, variables par défaut, tâches et gestionnaires). Pour plus d'informations, consultez la documentation Ansible pour "Rôles".

Les rôles sont souvent distribués dans le cadre d'une collection Ansible contenant des rôles et des modules associés. Donc, ces playbooks importent principalement plusieurs rôles distribués dans les différentes collections NetApp E-Series Ansible.

> Actuellement, au moins deux éléments de base (quatre nœuds de fichiers) sont nécessaires pour déployer BeeGFS, à moins qu'un périphérique quorum distinct soit configuré comme un disjoncteur d'attache pour limiter les problèmes lors de l'établissement du quorum avec un cluster à deux nœuds.

Étapes

1. Créer un nouveau `playbook.yml` classez et incluez les éléments suivants :

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
  gather_facts: false
  collections:
```

```
    - netapp_eseries.santricity
  tasks:
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
```

```
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

> Ce PlayBook s'exécute `pre_tasks` Vérifiez que Python 3 est installé sur les nœuds de fichiers et vérifiez que les balises Ansible fournies sont prises en charge.

2. Utilisez le `ansible-playbook` Commande avec les fichiers d'inventaire et de PlayBook lorsque vous êtes prêt à déployer BeeGFS.

   Le déploiement va s'exécuter tout `pre_tasks`, Puis demander confirmation de l'utilisateur avant de poursuivre le déploiement BeeGFS.

   Exécuter la commande suivante en réglant le nombre de fourches selon les besoins (voir la remarque ci-dessous) :

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

> En particulier pour les déploiements plus importants, en remplaçant le nombre par défaut de fourches (5) à l'aide du `forks` Paramètre recommandé pour augmenter le nombre d'hôtes qu'Ansible configure en parallèle. (Pour plus d'informations, voir "Réglage des performances Ansible" et "Contrôle de l'exécution de PlayBook".) Le paramètre de valeur maximale dépend de la puissance de traitement disponible sur le nœud de contrôle Ansible. L'exemple ci-dessus de 20 a été exécuté sur un nœud de contrôle Ansible virtuel avec 4 processeurs (Intel® Xeon® Gold 6146 CPU à 3,20 GHz).

Selon la taille du déploiement et les performances réseau entre le nœud de contrôle Ansible et les nœuds de fichier et bloc BeeGFS, la durée de déploiement peut varier.

Configurer les clients BeeGFS

Vous devez installer et configurer le client BeeGFS sur tous les hôtes qui doivent accéder au système de fichiers BeeGFS, comme les nœuds de calcul ou les nœuds GPU. Pour cette tâche, vous pouvez utiliser Ansible et la collection BeeGFS.

Étapes

1. Si nécessaire, configurez une connexion SSH sans mot de passe depuis le nœud de contrôle Ansible vers chacun des hôtes que vous souhaitez configurer comme clients BeeGFS :

   ```
   ssh-copy-id <user>@<HOSTNAME_OR_IP>
   ```

2. Sous `host_vars/`, Créez un fichier pour chaque client BeeGFS nommé `<HOSTNAME>.yml` avec le contenu suivant, en renseignant le texte de l'espace réservé contenant les informations correctes pour votre environnement :

   ```
   # BeeGFS Client
   ansible_host: <MANAGEMENT_IP>
   # OPTIONAL: If you want to use the NetApp E-Series Host Collection's
   IPoIB role to configure InfiniBand interfaces for clients to connect to
   BeeGFS file systems:
   eseries_ipoib_interfaces:
     - name: <INTERFACE>
       address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
     - name: <INTERFACE>0
       address: <IP>/<SUBNET_MASK>
   ```

   Actuellement, deux interfaces InfiniBand doivent être configurées sur chaque client, une dans chacun des deux sous-réseaux de stockage IPoIB. Si vous utilisez les exemples de sous-réseaux et les plages recommandées pour chaque service BeeGFS répertorié ici, les clients doivent avoir une interface configurée dans la plage de `100.127.1. 0` à `100.127.99.255` et l'autre dans `100.128.1. 0` à `100.128. 99.255`.

3. Créez un nouveau fichier `client_inventory.yml`, puis remplissez les paramètres suivants en haut :

```
# BeeGFS client inventory.
all:
  vars:
    ansible_ssh_user: <USER> # This is the user Ansible should use to
connect to each client.
    ansible_become_password: <PASSWORD> # This is the password Ansible
will use for privilege escalation, and requires the ansible_ssh_user be
root, or have sudo privileges.
The defaults set by the BeeGFS HA role are based on the testing
performed as part of this NetApp Verified Architecture and differ from
the typical BeeGFS client defaults.
```

> Ne stockez pas les mots de passe en texte brut. Utilisez plutôt Ansible Vault (consultez la documentation Ansible pour "Cryptage de contenu avec Ansible Vault") ou utilisez l' `--ask -become-pass` option lors de l'exécution du manuel de vente.

4. Dans le `client_inventory.yml` Fichier, répertorie tous les hôtes qui doivent être configurés comme clients BeeGFS sous `beegfs_clients` Définissez ensuite toute configuration supplémentaire requise pour générer le module de noyau client BeeGFS.

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

Lorsque vous utilisez les pilotes OFED Mellanox, assurez-vous que `beegfs_client_ofed_include_path` Indique le bon « chemin d'accès à l'en-tête » pour votre installation Linux. Pour plus d'informations, consultez la documentation BeeGFS pour "Prise en charge de RDMA".

5. Dans le `client_inventory.yml` Fichier, répertorie les systèmes de fichiers BeeGFS que vous souhaitez monter au bas de tout ce qui a été défini précédemment `vars`.

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

> Le `beegfs_client_config` représente les paramètres testés. Reportez-vous à la documentation fournie avec le `netapp_eseries.beegfs` collection `beegfs_client` rôle pour une vue d'ensemble complète de toutes les options. Cela inclut le montage de plusieurs systèmes de fichiers BeeGFS ou le montage du même système de fichiers BeeGFS plusieurs fois.

6. Créer un nouveau `client_playbook.yml` puis remplissez les paramètres suivants :

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> Ignorer l'importation du `netapp_eseries.host` collecte et `ipoib` Rôle si vous avez déjà installé les pilotes IB/RDMA requis et configuré les adresses IP sur les interfaces IPoIB appropriées.

7. Pour installer et construire le client et monter BeeGFS, exécutez la commande suivante :

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. Avant de placer le système de fichiers BeeGFS en production, nous vous recommandons **fortement** de vous connecter à n'importe quel client et de l'exécuter `beegfs-fsck --checkfs` afin de garantir que tous les nœuds sont accessibles et qu'aucun problème n'est signalé.

Informations sur le copyright

Copyright © 2022 NetApp, Inc. Tous droits réservés. Imprimé aux États-Unis. Aucune partie de ce document protégé par copyright ne peut être reproduite sous quelque forme que ce soit ou selon quelque méthode que ce soit (graphique, électronique ou mécanique, notamment par photocopie, enregistrement ou stockage dans un système de récupération électronique) sans l'autorisation écrite préalable du détenteur du droit de copyright.

Les logiciels dérivés des éléments NetApp protégés par copyright sont soumis à la licence et à l'avis de non-responsabilité suivants :

CE LOGICIEL EST FOURNI PAR NETAPP « EN L'ÉTAT » ET SANS GARANTIES EXPRESSES OU TACITES, Y COMPRIS LES GARANTIES TACITES DE QUALITÉ MARCHANDE ET D'ADÉQUATION À UN USAGE PARTICULIER, QUI SONT EXCLUES PAR LES PRÉSENTES. EN AUCUN CAS NETAPP NE SERA TENU POUR RESPONSABLE DE DOMMAGES DIRECTS, INDIRECTS, ACCESSOIRES, PARTICULIERS OU EXEMPLAIRES (Y COMPRIS L'ACHAT DE BIENS ET DE SERVICES DE SUBSTITUTION, LA PERTE DE JOUISSANCE, DE DONNÉES OU DE PROFITS, OU L'INTERRUPTION D'ACTIVITÉ), QUELLES QU'EN SOIENT LA CAUSE ET LA DOCTRINE DE RESPONSABILITÉ, QU'IL S'AGISSE DE RESPONSABILITÉ CONTRACTUELLE, STRICTE OU DÉLICTUELLE (Y COMPRIS LA NÉGLIGENCE OU AUTRE) DÉCOULANT DE L'UTILISATION DE CE LOGICIEL, MÊME SI LA SOCIÉTÉ A ÉTÉ INFORMÉE DE LA POSSIBILITÉ DE TELS DOMMAGES.

NetApp se réserve le droit de modifier les produits décrits dans le présent document à tout moment et sans préavis. NetApp décline toute responsabilité découlant de l'utilisation des produits décrits dans le présent document, sauf accord explicite écrit de NetApp. L'utilisation ou l'achat de ce produit ne concède pas de licence dans le cadre de droits de brevet, de droits de marque commerciale ou de tout autre droit de propriété intellectuelle de NetApp.

Le produit décrit dans ce manuel peut être protégé par un ou plusieurs brevets américains, étrangers ou par une demande en attente.

LÉGENDE DE RESTRICTION DES DROITS : L'utilisation, la duplication ou la divulgation par le gouvernement sont sujettes aux restrictions énoncées dans le sous-paragraphe (b)(3) de la clause Rights in Technical Data-Noncommercial Items du DFARS 252.227-7013 (février 2014) et du FAR 52.227-19 (décembre 2007).

Les données contenues dans les présentes se rapportent à un produit et/ou service commercial (tel que défini par la clause FAR 2.101). Il s'agit de données propriétaires de NetApp, Inc. Toutes les données techniques et tous les logiciels fournis par NetApp en vertu du présent Accord sont à caractère commercial et ont été exclusivement développés à l'aide de fonds privés. Le gouvernement des États-Unis dispose d'une licence limitée irrévocable, non exclusive, non cessible, non transférable et mondiale. Cette licence lui permet d'utiliser uniquement les données relatives au contrat du gouvernement des États-Unis d'après lequel les données lui ont été fournies ou celles qui sont nécessaires à son exécution. Sauf dispositions contraires énoncées dans les présentes, l'utilisation, la divulgation, la reproduction, la modification, l'exécution, l'affichage des données sont interdits sans avoir obtenu le consentement écrit préalable de NetApp, Inc. Les droits de licences du Département de la Défense du gouvernement des États-Unis se limitent aux droits identifiés par la clause 252.227-7015(b) du DFARS (février 2014).

Informations sur les marques commerciales

NETAPP, le logo NETAPP et les marques citées sur le site http://www.netapp.com/TM sont des marques déposées ou des marques commerciales de NetApp, Inc. Les autres noms de marques et de produits sont des marques commerciales de leurs propriétaires respectifs.